Name:- Samruddhi Sangram Yadav
Roll no. B560

Title:-
Use Autoencoder to implement anomaly detection.
Build the model by using:
a. Import required libraries
b. Upload / access the dataset
c. Encoder converts it into latent representation
d. Decoder networks convert it back to the original input
e. Compile the models with Optimizer, Loss, and Evaluation Metrics

CODE:-

```
# A. Import Required Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from tensorflow.keras import layers, Model
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score

print("Libraries imported successfully!")

# B. Access the Dataset
url = "https://storage.googleapis.com/download.tensorflow.org/data/ecg.csv"
df = pd.read_csv(url, header=None)

data = df.iloc[:, :-1].values
labels = df.iloc[:, -1].values

print("\nDataset Shape:", df.shape)
print("Feature Shape:", data.shape)
print("Labels:", np.unique(labels))

# Split into Training and Testing Data
x_train, x_test, y_train, y_test = train_test_split(
    data, labels, test_size=0.2, random_state=21
)

# Normalize the Data
minimum = x_train.min()
maximum = x_train.max()
```

```
x_train = ((x_train - minimum) / (maximum - minimum)).astype("float32")
x_test = ((x_test - minimum) / (maximum - minimum)).astype("float32")

# Use only normal samples for training
normal_train = x_train[y_train == 1]
normal_test = x_test[y_test == 1]

print("\nNormal Training Samples:", normal_train.shape)

# C. Build the Encoder
encoder = tf.keras.Sequential([
    layers.Input(shape=(140,)),
    layers.Dense(32, activation="relu"),
    layers.Dense(16, activation="relu"),
    layers.Dense(8, activation="relu")
], name="Encoder")

# D. Build the Decoder
decoder = tf.keras.Sequential([
    layers.Input(shape=(8,)),
    layers.Dense(16, activation="relu"),
    layers.Dense(32, activation="relu"),
    layers.Dense(140, activation="sigmoid")
], name="Decoder")

# Combine Encoder and Decoder
inputs = layers.Input(shape=(140,))
encoded = encoder(inputs)
decoded = decoder(encoded)

autoencoder = Model(inputs, decoded, name="Autoencoder")

# E. Compile the Model
autoencoder.compile(
    optimizer="adam",
    loss="mse",
    metrics=["mae"]
)

print("\nAutoencoder Model Summary:")
autoencoder.summary()

# Train the Autoencoder on Normal Data
print("\nTraining the Autoencoder...")

history = autoencoder.fit(
```

```
    normal_train,
    normal_train,
    epochs=20,
    batch_size=128,
    validation_data=(normal_test, normal_test),
    shuffle=True,
    verbose=1
)

print("\nTraining Completed Successfully!")

# Plot Training and Validation Loss
plt.figure(figsize=(8, 4))
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Mean Squared Error")
plt.title("Autoencoder Training Loss")
plt.legend()
plt.grid(True)
plt.show()

# Detect Anomalies Using Reconstruction Error
train_reconstructed = autoencoder.predict(normal_train, verbose=0)

train_errors = np.mean(
    np.abs(normal_train - train_reconstructed), axis=1
)

# Define Anomaly Threshold
threshold = np.mean(train_errors) + np.std(train_errors)
print("\nAnomaly Threshold:", round(float(threshold), 6))

# Reconstruct Test Data
test_reconstructed = autoencoder.predict(x_test, verbose=0)

test_errors = np.mean(
    np.abs(x_test - test_reconstructed), axis=1
)

# 1 = Anomaly, 0 = Normal
predictions = (test_errors > threshold).astype(int)

# Evaluate the Model
actual_anomalies = (y_test == 0).astype(int)
```

```
accuracy = accuracy_score(actual_anomalies, predictions)
precision = precision_score(
    actual_anomalies, predictions, zero_division=0
)
recall = recall_score(
    actual_anomalies, predictions, zero_division=0
)

print("\n--- Anomaly Detection Results ---")
print("Accuracy :", round(accuracy * 100, 2), "%")
print("Precision:", round(precision * 100, 2), "%")
print("Recall   :", round(recall * 100, 2), "%")

print("\nFirst 10 Predictions (1 = Anomaly, 0 = Normal):")
print(predictions[:10])

# Visualize Original and Reconstructed Signals
plt.figure(figsize=(10, 4))
plt.plot(x_test[0], label="Original Input")
plt.plot(test_reconstructed[0], label="Reconstructed Output")
plt.title("Original vs Reconstructed Signal")
plt.xlabel("Feature Index")
plt.ylabel("Normalized Value")
plt.legend()
plt.grid(True)
plt.show()

# Plot Reconstruction Error Distribution
plt.figure(figsize=(8, 4))
plt.hist(test_errors, bins=40)
plt.axvline(
    threshold, linestyle="--", label="Anomaly Threshold"
)
plt.xlabel("Reconstruction Error")
plt.ylabel("Number of Samples")
plt.title("Anomaly Detection Results")
plt.legend()
plt.grid(True)
plt.show()

print("\nPractical Executed Successfully!")
```

OUTPUT:-

```
%runfile C:/Users/CD/.spyder-py3/test.py –wdir
Libraries imported successfully!
```

Dataset Shape: (4998, 141)
Feature Shape: (4998, 140)
Labels: [0. 1.]

Normal Training Samples: (2359, 140)

Autoencoder Model Summary:
Model: "Autoencoder"

Layer (type)	Output Shape	Param #
input_layer_2 (InputLayer)	(None, 140)	0
Encoder (Sequential)	(None, 8)	5,176
Decoder (Sequential)	(None, 140)	5,308

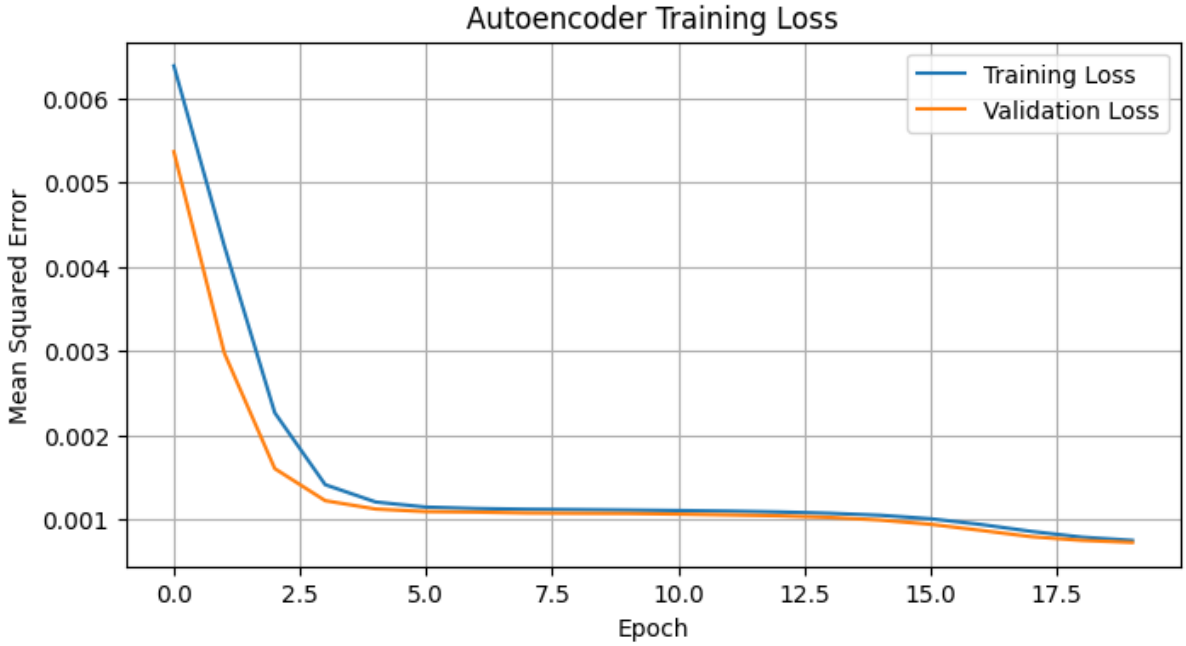
Total params: 10,484 (40.95 KB)
Trainable params: 10,484 (40.95 KB)
Non-trainable params: 0 (0.00 B)

Training the Autoencoder...
Epoch 1/20
19/19 **5s** 66ms/step - loss: 0.0064 - mae: 0.0554 - val_loss: 0.0054 - val_mae: 0.0501
Epoch 2/20
19/19 **1s** 36ms/step - loss: 0.0042 - mae: 0.0436 - val_loss: 0.0030 - val_mae: 0.0355
Epoch 3/20
19/19 **1s** 31ms/step - loss: 0.0023 - mae: 0.0302 - val_loss: 0.0016 - val_mae: 0.0248
Epoch 4/20
19/19 **1s** 27ms/step - loss: 0.0014 - mae: 0.0232 - val_loss: 0.0012 - val_mae: 0.0217
Epoch 5/20

19/19 ━━━━━━━━━━━━━━━━━━━━ **1s** 32ms/step - loss: 0.0012 - mae: 0.0216 - val_loss: 0.0011 - val_mae: 0.0208
Epoch 6/20
19/19 ━━━━━━━━━━━━━━━━━━━━ **1s** 14ms/step - loss: 0.0011 - mae: 0.0210 - val_loss: 0.0011 - val_mae: 0.0206
Epoch 7/20
19/19 ━━━━━━━━━━━━━━━━━━━━ **0s** 16ms/step - loss: 0.0011 - mae: 0.0209 - val_loss: 0.0011 - val_mae: 0.0206
Epoch 8/20
19/19 ━━━━━━━━━━━━━━━━━━━━ **0s** 11ms/step - loss: 0.0011 - mae: 0.0208 - val_loss: 0.0011 - val_mae: 0.0205
Epoch 9/20
19/19 ━━━━━━━━━━━━━━━━━━━━ **0s** 14ms/step - loss: 0.0011 - mae: 0.0208 - val_loss: 0.0011 - val_mae: 0.0204
Epoch 10/20
19/19 ━━━━━━━━━━━━━━━━━━━━ **0s** 12ms/step - loss: 0.0011 - mae: 0.0207 - val_loss: 0.0011 - val_mae: 0.0203
Epoch 11/20
19/19 ━━━━━━━━━━━━━━━━━━━━ **0s** 18ms/step - loss: 0.0011 - mae: 0.0206 - val_loss: 0.0011 - val_mae: 0.0203
Epoch 12/20
19/19 ━━━━━━━━━━━━━━━━━━━━ **1s** 12ms/step - loss: 0.0011 - mae: 0.0206 - val_loss: 0.0011 - val_mae: 0.0203
Epoch 13/20
19/19 ━━━━━━━━━━━━━━━━━━━━ **0s** 18ms/step - loss: 0.0011 - mae: 0.0205 - val_loss: 0.0010 - val_mae: 0.0201
Epoch 14/20
19/19 ━━━━━━━━━━━━━━━━━━━━ **0s** 6ms/step - loss: 0.0011 - mae: 0.0204 - val_loss: 0.0010 - val_mae: 0.0199
Epoch 15/20
19/19 ━━━━━━━━━━━━━━━━━━━━ **0s** 6ms/step - loss: 0.0011 - mae: 0.0202 - val_loss: 9.9309e-04 - val_mae: 0.0196
Epoch 16/20
19/19 ━━━━━━━━━━━━━━━━━━━━ **0s** 6ms/step - loss: 0.0010 - mae: 0.0198 - val_loss: 9.4262e-04 - val_mae: 0.0191
Epoch 17/20
19/19 ━━━━━━━━━━━━━━━━━━━━ **0s** 6ms/step - loss: 9.4060e-04 - mae: 0.0190 - val_loss: 8.7053e-04 - val_mae: 0.0184
Epoch 18/20
19/19 ━━━━━━━━━━━━━━━━━━━━ **0s** 6ms/step - loss: 8.5845e-04 - mae: 0.0183 - val_loss: 7.9519e-04 - val_mae: 0.0175
Epoch 19/20
19/19 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - loss: 7.9014e-04 - mae: 0.0175 - val_loss: 7.5214e-04 - val_mae: 0.0171
Epoch 20/20

19/19 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **0s** 6ms/step - loss: 7.5183e-04 - mae: 0.0170 - val_loss: 7.2716e-04 - val_mae: 0.0167

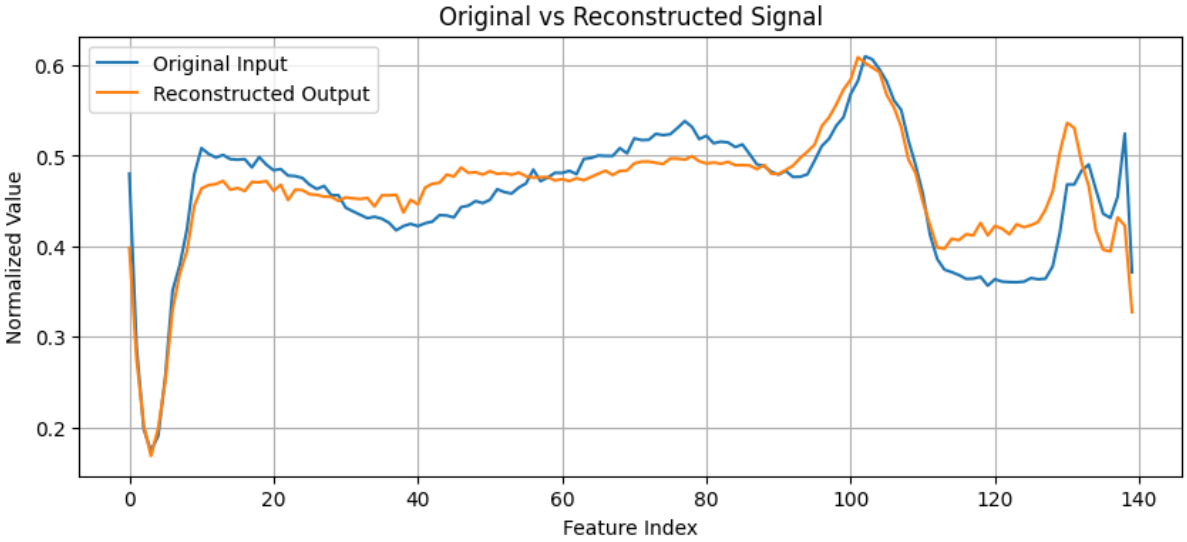
Training Completed Successfully!

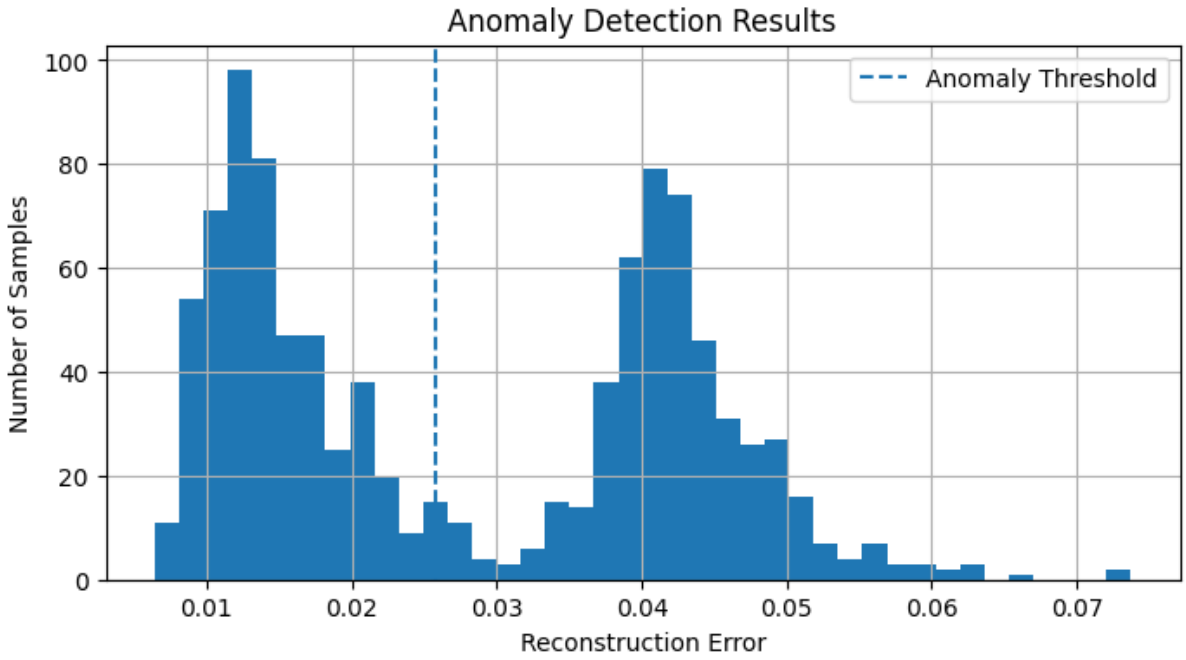


Anomaly Threshold: 0.025723

--- Anomaly Detection Results ---
Accuracy : 94.2 %
Precision: 88.98 %
Recall : 99.09 %

First 10 Predictions (1 = Anomaly, 0 = Normal):
[1 1 0 0 1 1 0 0 1 0]




Anomaly Detection Results
Anomaly Threshold
Number of Samples
Reconstruction Error
0
20
40
60
80
100
0.01
0.02
0.03
0.04
0.05
0.06
0.07